SIMPLE ROUTING IN REACT

Learn React Router the Easy Way

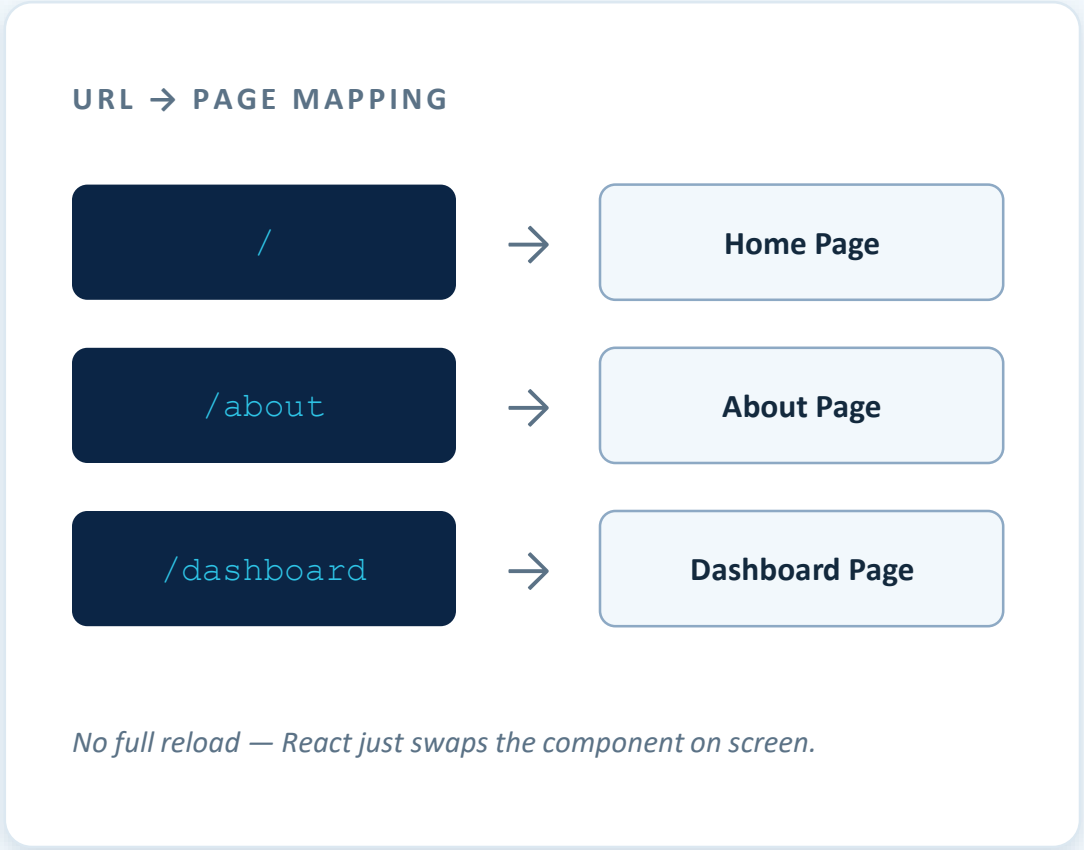
From basic page navigation to building a real Dashboard Layout — explained step by step, with simple code examples.

Frontend Development • React.js • Lecture Deck

WHAT IS “ROUTING”?

Showing the right page, without reloading the site

- In a normal website, every link click reloads the whole page from the server.
- React apps are “Single Page Applications” (SPA) — only ONE HTML page ever loads.
- Routing lets us change what's shown on screen based on the URL — without a real reload.
- In React, this is done using a library called React Router (react-router-dom).



REACT ROUTER BASICS

Four building blocks you must know

1

BrowserRouter

Wraps your whole app. It turns on routing and reads the browser's URL.

2

Routes

A container that holds all your possible <Route> paths, like a switch box.

3

Route

One single rule: “when the URL matches this path, show this component.”

4

Link / NavLink

A replacement for <a> tags — changes the URL without reloading the page.

GETTING STARTED

Step 1: Install & wrap your app

1

Install the package

Run this once in your project folder.

2

Import the pieces

Bring in BrowserRouter, Routes and Route from the library.

3

Wrap <App />

Put <BrowserRouter> around your whole application, usually in main.jsx / index.js.

```
// 1. install
npm install react-router-dom

// 2 & 3. wrap your app
import {
  BrowserRouter
} from "react-router-dom";

<BrowserRouter>
  <App />
</BrowserRouter>
```

BASIC EXAMPLE

Step 2: Define your routes

```
function App() {
  return (
    <Routes>
      <Route path="/"
        element={<Home />} />
      <Route path="/about"
        element={<About />} />
      <Route path="/dashboard"
        element={<Dashboard />} />
    </Routes>
  );
}
```

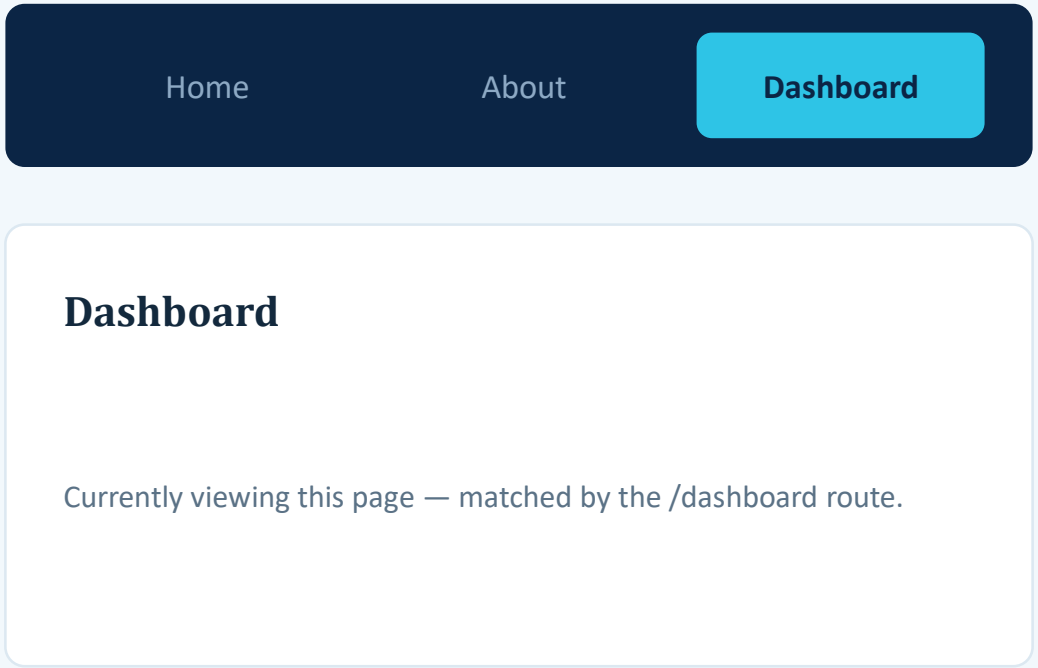
- path = the URL pattern to match
- element = which component to show
- Every <Route> is one page in your app
- React checks the URL top to bottom and renders the first match

NAVIGATION

Step 3: Move between pages

```
import { Link } from
  "react-router-dom";

<Link to="/">Home</Link>
<Link to="/about">About</Link>
<Link to="/dashboard">
  Dashboard
</Link>
```

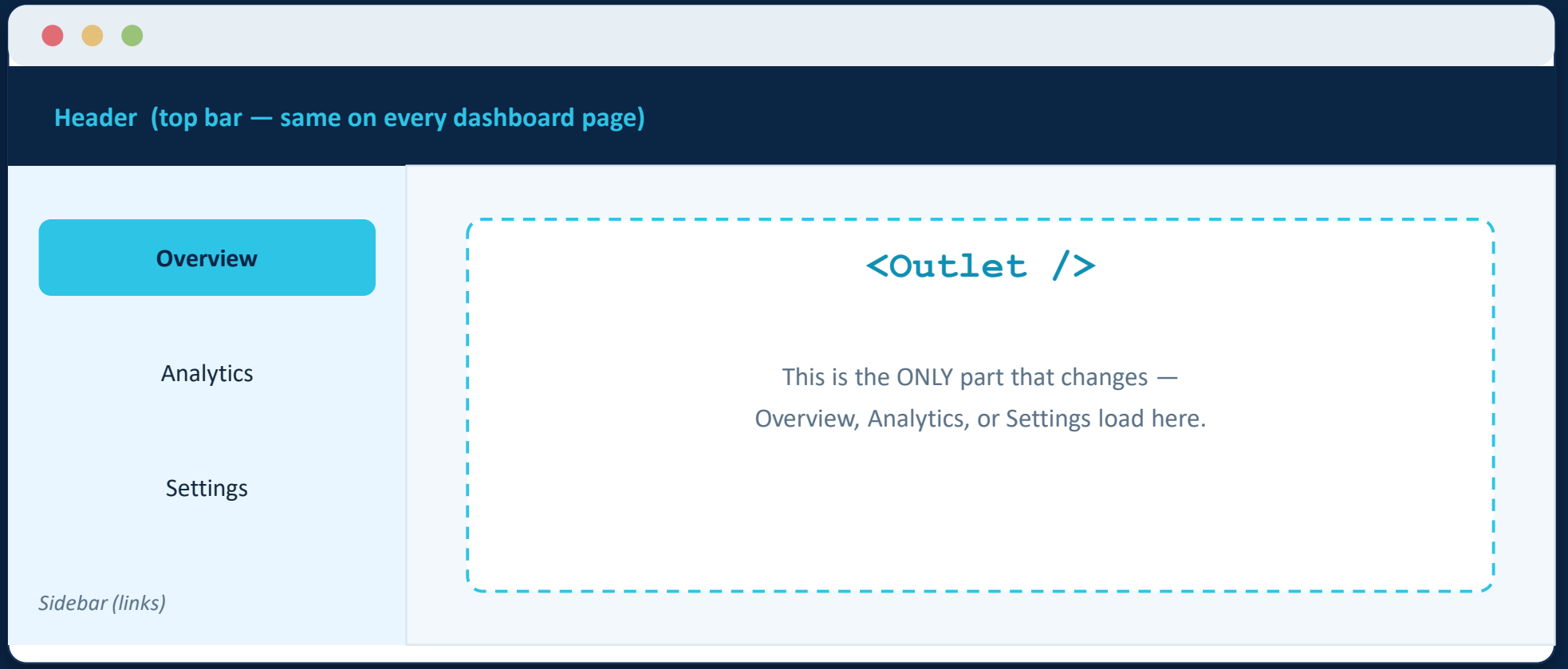


Link never reloads the browser — it just swaps the matching <Route>. NavLink is the same, but adds an “active” style automatically.

PUTTING IT TOGETHER

Example: a Dashboard Layout

A dashboard has parts that never change (Sidebar, Header) and a middle area that changes per page. Routing lets us reuse the same layout for every dashboard page.



PUTTING IT TOGETHER

Dashboard Layout — the code

```
// nested routes share one layout
<Route path="/dashboard"
  element={<DashboardLayout />}>
  <Route index
    element={<Overview />} />
  <Route path="analytics"
    element={<Analytics />} />
  <Route path="settings"
    element={<Settings />} />
</Route>
```

- DashboardLayout renders the Sidebar + Header once
- Inside it, <Outlet /> marks where the child page appears
- "index" route = the default page (e.g. /dashboard)
- "analytics" / "settings" become /dashboard/analytics, etc.
- Result: one layout, many interchangeable pages

KEY TAKEAWAYS

Simple Routing, Summed Up

1	**BrowserRouter**	Turns on routing for the whole app
2	**Routes + Route**	Map a URL path to a component
3	**Link / NavLink**	Navigate pages with no reload
4	**Outlet**	Marks where child pages render inside a layout
5	**Dashboard pattern**	One layout (Sidebar + Header) reused by many nested routes

Thank You — Questions?